

Lab02.Reis

Stefany Reis

Laboratório 2: Processamento de Bases de Dados em Lote

```
library(tidyverse)
```

Warning: pacote 'tidyverse' foi compilado no R versão 4.5.3

Warning: pacote 'ggplot2' foi compilado no R versão 4.5.3

Warning: pacote 'dplyr' foi compilado no R versão 4.5.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.1.5
v forcats    1.0.1      v stringr    1.5.2
v ggplot2    4.0.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.1.0
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?
2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- b. Remova observações que tenham valores faltantes em campos de interesse;
- c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
- e. A função deve receber apenas dois argumentos: i.input: o conjunto de dados (referente ao lote em questão); ii.pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
library(readr)

getStats <- function(input, pos) {

  input %>%
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%
    filter(!is.na(MONTH),
           !is.na(DAY),
           !is.na(AIRLINE),
           !is.na(ARRIVAL_DELAY)) %>%
    group_by(MONTH, DAY, AIRLINE) %>%
    summarise(
      n = n(),
      n_atrasados = sum(ARRIVAL_DELAY > 10),
      .groups = "drop"
    )
}
```

3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;
 - c. Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
library(readr)

callback <- DataFrameCallback$new(getStats)
```

```
dados_lotes <- read_csv_chunked(
  "flights.csv.zip",
  callback = callback,
  chunk_size = 100000,
  col_types = cols(
    .default = col_skip(),
    MONTH = col_integer(),
    DAY = col_integer(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

head(dados_lotes)
```

```
# A tibble: 6 x 5
  MONTH DAY AIRLINE      n n_atrasados
  <int> <int> <chr>   <int>      <int>
1     1     1 AA     1379        514
2     1     1 DL     1558        132
3     1     1 UA     1269        367
4     1     1 US     1028        201
5     1     2 AA     1508        649
6     1     2 DL     2261        406
```

4. Crie uma função chamada `computeStats` que:

- Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- Retorne as informações em um tibble contendo apenas as seguintes colunas: i.Cia: sigla da companhia aérea; ii.Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); iii.Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats <- function(stats) {

  stats %>%
    group_by(MONTH, DAY, AIRLINE) %>%
    summarise(
      n = sum(n),
      n_atrasados = sum(n_atrasados),
      .groups = "drop"
    )
}
```

```

    ) %>%
    mutate(
      Perc = n_atrasados / n,
      Data = as.Date(
        paste(2015, MONTH, DAY, sep = "-")
      ),
      Cia = AIRLINE
    ) %>%
    select(Cia, Data, Perc)
}

resultado <- computeStats(dados_lotes)
head(resultado)

```

```

# A tibble: 6 x 3
  Cia   Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a. Instale e carregue os pacotes ggcal e ggplot2.
- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
- ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
- iii. Retornar para o usuário a base do calendário criada acima.
- iv. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do ggplot2. Lembre-se de adicionar um título utilizando o comando ggtitle. Por exemplo, cXX + pal + ggtitle("Título").

```
library(ggplot2)
library(ggcal)

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

baseCalendario <- function(stats, cia) {

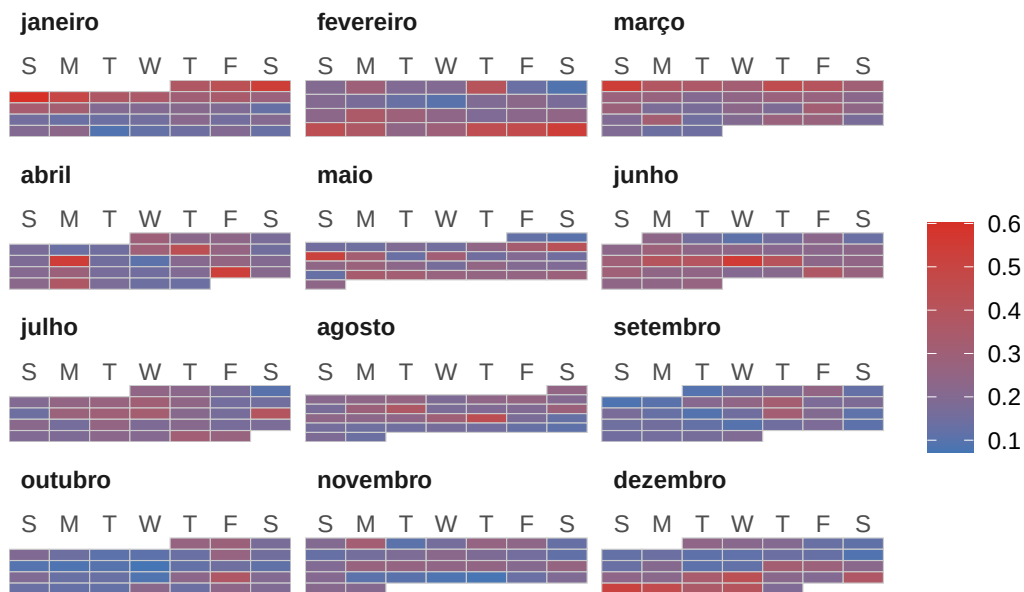
  dados <- stats %>%
    filter(Cia == cia)

  ggcal(
    dados$Data,
    dados$Perc
  )
}

cAA <- baseCalendario(resultado, "AA")
cDL <- baseCalendario(resultado, "DL")
cUA <- baseCalendario(resultado, "UA")
cUS <- baseCalendario(resultado, "US")

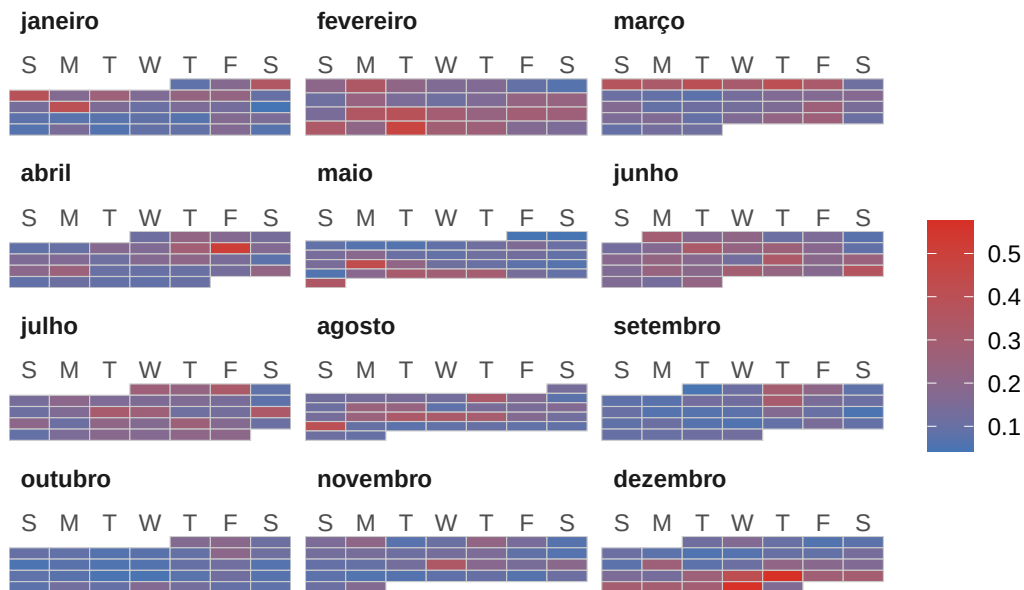
cAA + pal + ggtitle("Percentual de atrasos - American Airlines (AA)")
```

Percentual de atrasos – American Airlines (AA)



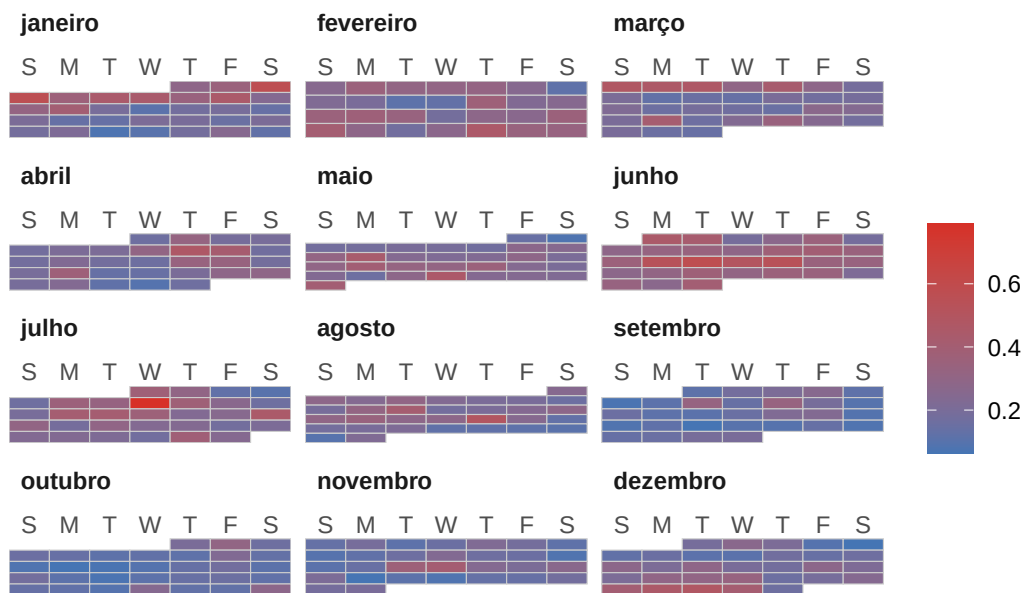
```
cDL + pal + ggtitle("Percentual de atrasos – Delta Airlines (DL)")
```

Percentual de atrasos – Delta Airlines (DL)



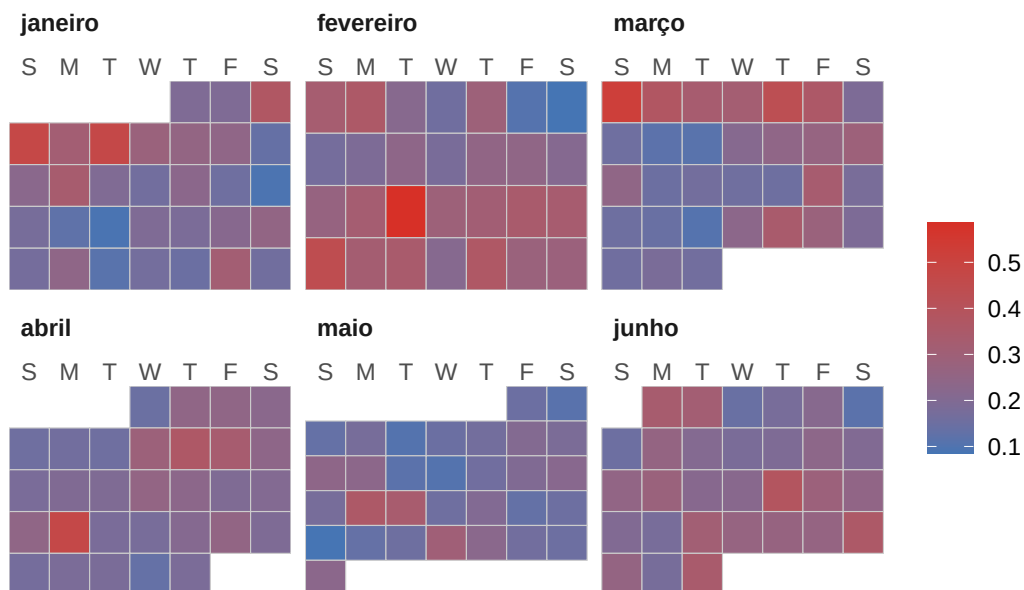
```
cUA + pal + ggtitle("Percentual de atrasos - United Airlines (UA)")
```

Percentual de atrasos – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de atrasos - US Airways (US)")
```

Percentual de atrasos – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-23 20:55:36 -03"
```